

CÁC THUẬT TOÁN TÌM KIẾM CÓ THÔNG TIN (INFORMED SEARCH)

CHƯƠNG 4, PHẦN 1–2, 4

Nội dung

- ◇ Tìm kiếm tốt nhất đầu tiên (Best-first search)
- ◇ Tìm kiếm A*
- ◇ Hàm Heuristic
- ◇ Leo đồi (Hill-climbing)
- ◇ Luyện kim mô phỏng (Simulated annealing)

Ôn tập: Tìm kiếm tổng quát

```
function GENERAL-SEARCH(problem, QUEUING-FN) returns một giải pháp, hoặc thất bại
nodes ← MAKE-QUEUE(MAKE-NODE(INITIAL-STATE[problem]))
loop do
  if nodes trống then return thất bại
  node ← REMOVE-FRONT(nodes)
  if GOAL-TEST[problem] áp dụng cho STATE(node) thành công then return node
  nodes ← QUEUING-FN(nodes, EXPAND(node, OPERATORS[problem]))
end
```

Một chiến lược được xác định bằng cách chọn *thứ tự phát triển nút*

Tìm kiếm tốt nhất đầu tiên (Best-first search)

Ý tưởng: sử dụng một *hàm đánh giá* cho mỗi nút

– ước tính mức độ “đáng mong muốn”

⇒ Phát triển nút chưa được phát triển có mức độ mong muốn cao nhất

Cài đặt:

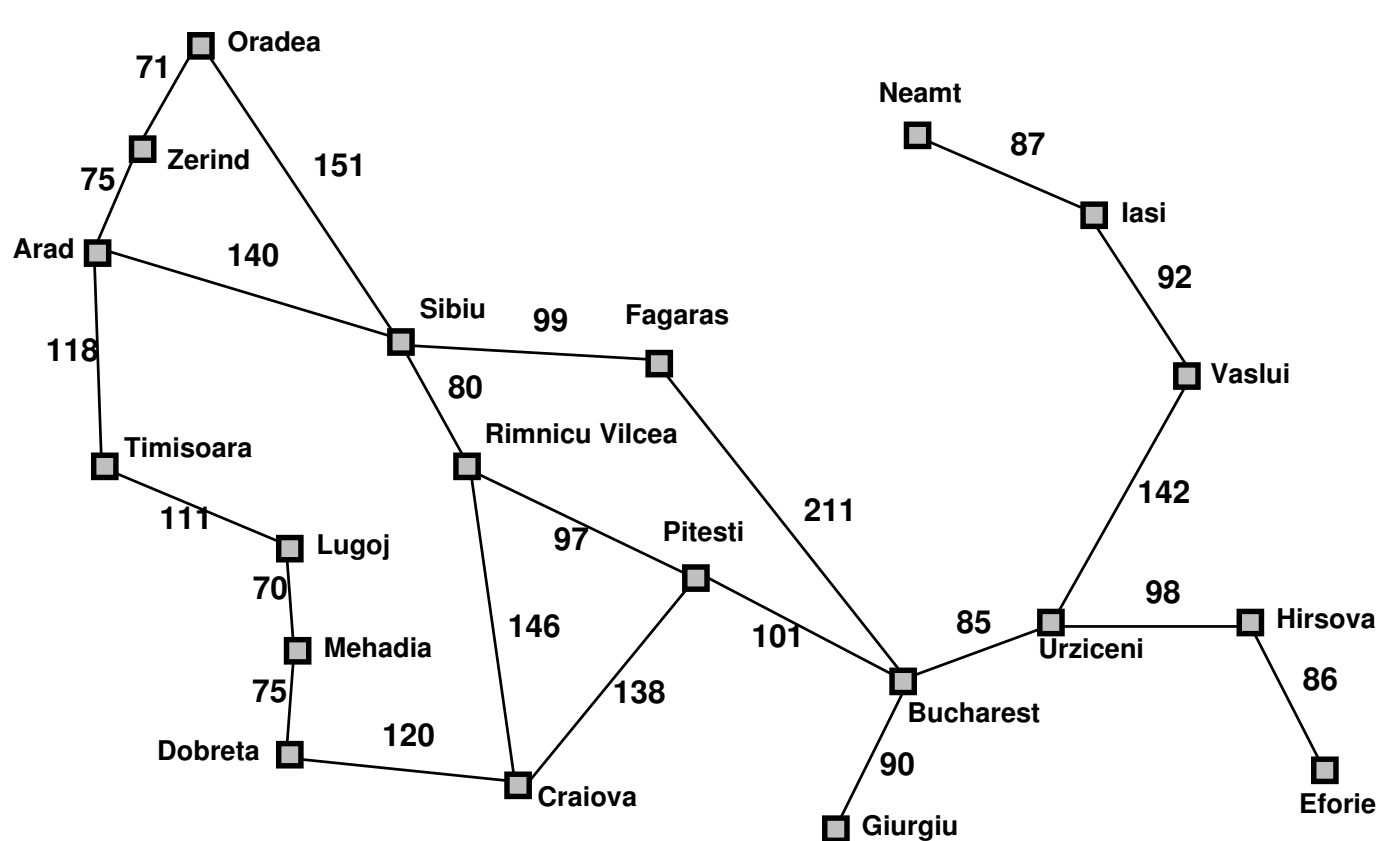
QUEUEINGFN = chèn các trạng thái kế tiếp theo thứ tự giảm dần của mức độ mong muốn

Các trường hợp đặc biệt:

tìm kiếm tham lam (greedy search)

tìm kiếm A^*

Bản đồ Romania với chi phí bước tính bằng km



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Tìm kiếm tham lam (Greedy search)

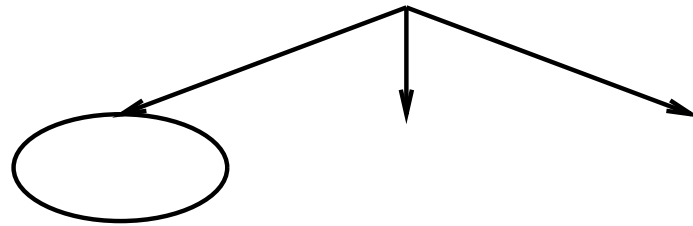
Hàm đánh giá $h(n)$ (hàm khám phá - heuristic)

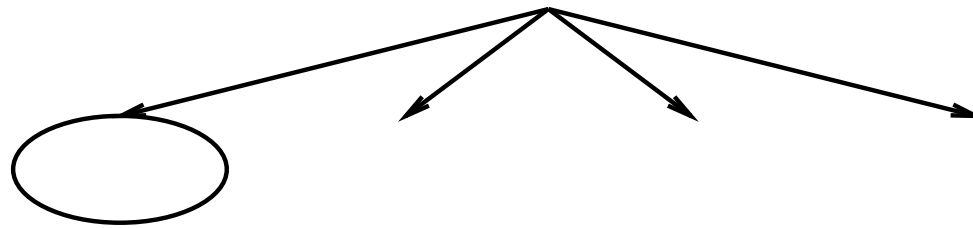
= ước lượng chi phí từ n đến *đích* (*goal*)

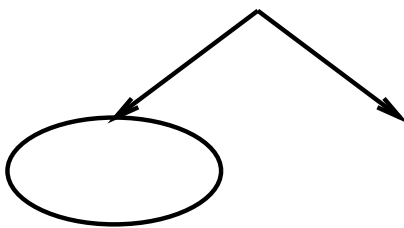
Ví dụ, $h_{\text{SLD}}(n)$ = khoảng cách đường chim bay từ n đến Bucharest

Tìm kiếm tham lam phát triển nút *có vẻ* gần đích nhất

Ví dụ tìm kiếm tham lam







Thuộc tính của tìm kiếm tham lam

- ⋈ Đầy đủ (Complete)??
- ⋈ Thời gian (Time)??
- ⋈ Không gian bộ nhớ (Space)??
- ⋈ Tối ưu (Optimal)??

Thuộc tính của tìm kiếm tham lam

◡ Đầy đủ?? Không – có thể bị mắc kẹt trong các vòng lặp, ví dụ:

Iasi $\rightarrow$ Neamt $\rightarrow$ Iasi $\rightarrow$ Neamt $\rightarrow$

Sẽ đầy đủ trong không gian hữu hạn nếu có kiểm tra trạng thái lặp

◡ Thời gian?? $O(b^m)$, nhưng một heuristic tốt có thể cải thiện đáng kể

◡ Không gian?? $O(b^m)$ — giữ tất cả các nút trong bộ nhớ

◡ Tối ưu?? Không

Tìm kiếm A^*

Ý tưởng: tránh phát triển các đường đi đã tốn kém

Hàm đánh giá $f(n) = g(n) + h(n)$

$g(n)$ = chi phí tính đến hiện tại để đạt đến n

$h(n)$ = chi phí ước tính đến đích từ n

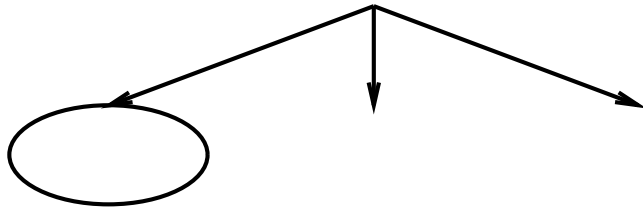
$f(n)$ = tổng chi phí ước tính của đường đi qua n đến đích

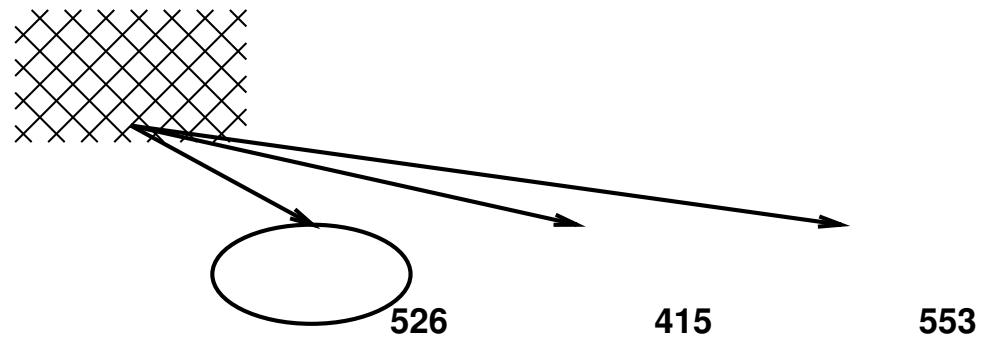
Tìm kiếm A^* sử dụng một heuristic *chấp nhận được* (admissible) nghĩa là, $h(n) \leq h^*(n)$ trong đó $h^*(n)$ là chi phí *thực tế* từ n .

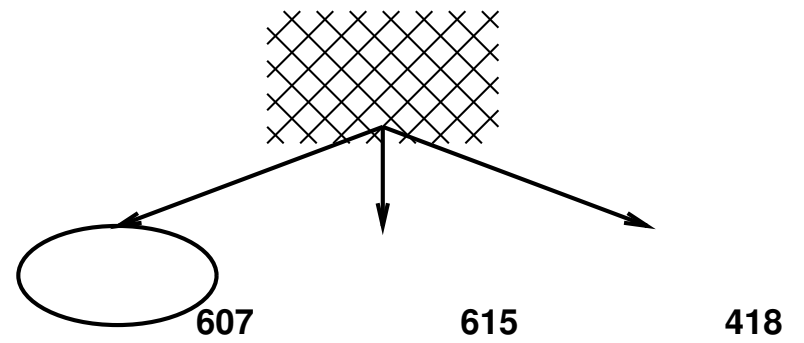
Ví dụ, $h_{\text{SLD}}(n)$ không bao giờ đánh giá quá cao khoảng cách đường bộ thực tế

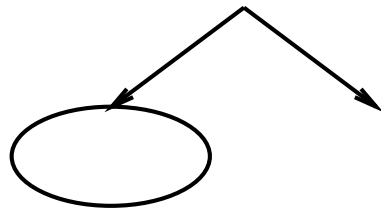
Định lý: Tìm kiếm A^* là tối ưu

Ví dụ tìm kiếm A^*



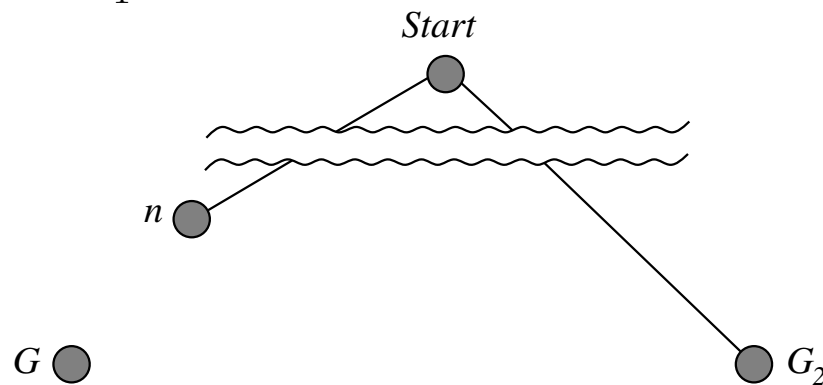






Tính tối ưu của A^* (chứng minh tiêu chuẩn)

Giả sử một nút đích không tối ưu G_2 đã được sinh ra và đang ở trong hàng đợi. Gọi n là một nút chưa được phát triển nằm trên đường đi ngắn nhất đến đích tối ưu G_1 .



$$\begin{aligned} f(G_2) &= g(G_2) && \text{vì } h(G_2) = 0 \\ &> g(G_1) && \text{vì } G_2 \text{ là không tối ưu} \\ &\geq f(n) && \text{vì } h \text{ là chấp nhận được} \end{aligned}$$

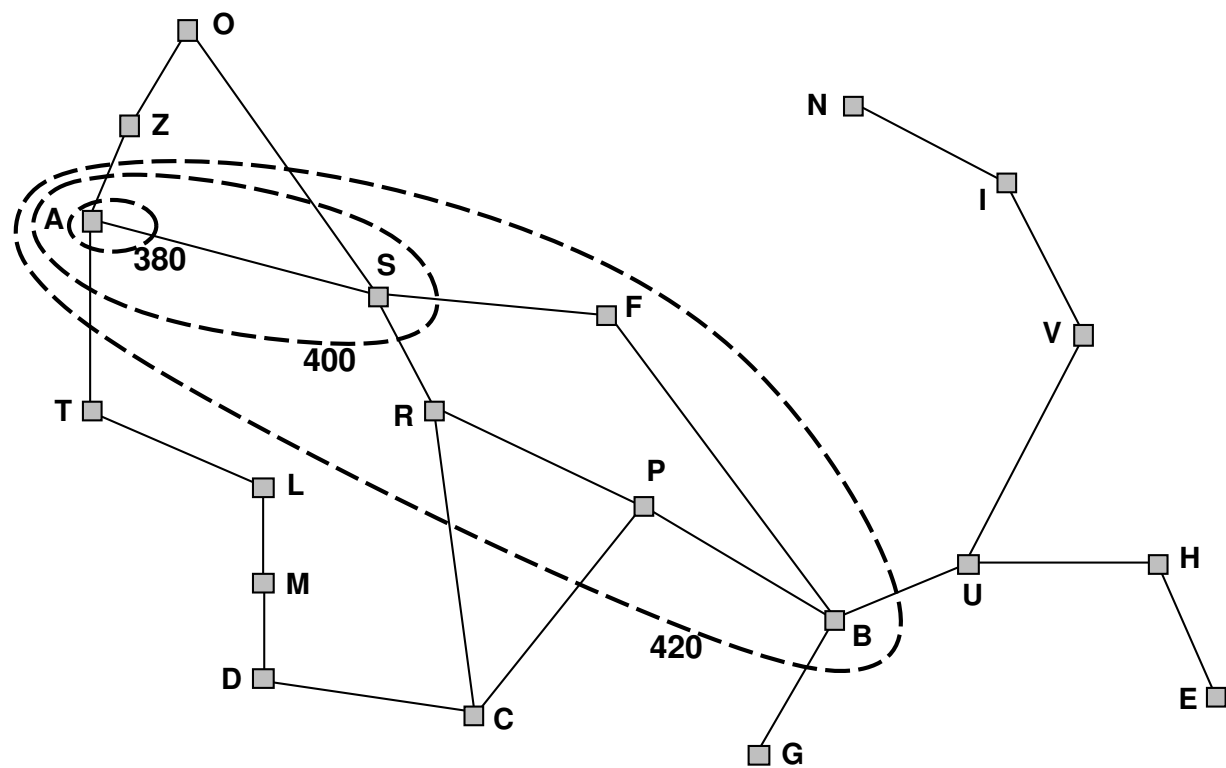
Vì $f(G_2) > f(n)$, A^* sẽ không bao giờ chọn G_2 để phát triển

Tính tối ưu của A^* (hữu ích hơn)

Bổ đề: A^* phát triển các nút theo thứ tự giá trị f tăng dần

Từ từ thêm các “đường đồng mức f ” của các nút (so sánh với tìm kiếm chiều rộng thêm các lớp)

Đường đồng mức i chứa tất cả các nút có $f = f_i$, trong đó $f_i < f_{i+1}$



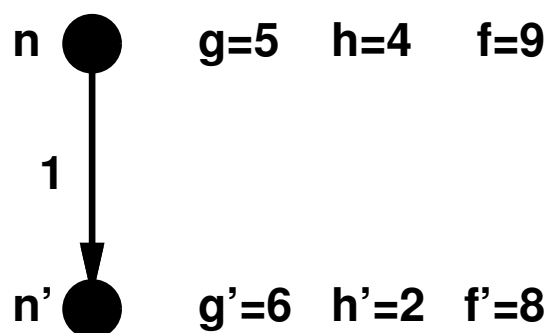
Thuộc tính của A^*

- ◌ Đầy đủ?? Có, trừ khi có vô số nút với $f \leq f(G)$
- ◌ Thời gian?? Hàm mũ theo [sai số tương đối của $h \times$ chiều dài của giải pháp]
- ◌ Không gian?? Giữ tất cả các nút trong bộ nhớ
- ◌ Tối ưu?? Có — không thể phát triển f_{i+1} cho đến khi f_i hoàn tất

Chứng minh bổ đề: Pathmax

Đối với một số heuristic chấp nhận được, f có thể *giảm* dọc theo một đường đi

Ví dụ, giả sử n' là một nút kế tiếp của n



Nhưng điều này ném bỏ thông tin!

$f(n) = 9 \Rightarrow$ chi phí thực sự của đường đi qua n là ≥ 9

Do đó chi phí thực sự của đường đi qua n' cũng ≥ 9

Điều chỉnh Pathmax cho A^* :

Thay vì $f(n') = g(n') + h(n')$, sử dụng $f(n') = \max(g(n') + h(n'), f(n))$

Với pathmax, f luôn không giảm dọc theo bất kỳ đường đi nào

Các heuristic chấp nhận được

Ví dụ, cho bài toán 8-puzzle (trò chơi xếp số 8 ô):

$h_1(n)$ = số ô đặt sai vị trí

$h_2(n)$ = tổng khoảng cách Manhattan

(tức là tổng số ô vuông từ vị trí hiện tại đến vị trí mong muốn của mỗi ô)

5	4	
6	1	8
7	3	2

Start State

1	2	3
8		4
7	6	5

Goal State

$h_1(S) = ??$

$h_2(S) = ??$

Các heuristic chấp nhận được

Ví dụ, cho bài toán 8-puzzle:

$h_1(n)$ = số ô đặt sai vị trí

$h_2(n)$ = tổng khoảng cách Manhattan

(tức là tổng số ô vuông từ vị trí hiện tại đến vị trí mong muốn của mỗi ô)

5	4	
6	1	8
7	3	2

Start State

1	2	3
8		4
7	6	5

Goal State

$$h_1(S) = ?? \quad 7$$

$$h_2(S) = ?? \quad 2+3+3+2+4+2+0+2 = 18$$

Sự vượt trội (Dominance)

Nếu $h_2(n) \geq h_1(n)$ với mọi n (cả hai đều chấp nhận được)
thì h_2 vượt trội h_1 và tốt hơn cho quá trình tìm kiếm

Chi phí tìm kiếm điển hình:

$d = 14$ IDS = 3,473,941 nút

$A^*(h_1) = 539$ nút

$A^*(h_2) = 113$ nút

$d = 14$ IDS = quá nhiều nút

$A^*(h_1) = 39,135$ nút

$A^*(h_2) = 1,641$ nút

Bài toán nới lỏng (Relaxed problems)

Các heuristic chấp nhận được có thể được suy ra từ chi phí giải pháp *chính xác* của phiên bản *nới lỏng* của bài toán

Nếu các quy tắc của bài toán 8-puzzle được nới lỏng để một ô có thể di chuyển *bất cứ nơi đâu*, thì $h_1(n)$ cho ta giải pháp ngắn nhất

Nếu các quy tắc được nới lỏng để một ô có thể di chuyển đến *bất kỳ ô kề cạnh nào*, thì $h_2(n)$ cho ta giải pháp ngắn nhất

Đối với bài toán Người giao hàng (TSP): gọi đường đi là *bất kỳ* cấu trúc nào nối tất cả các thành phố

$\Rightarrow$ heuristic là cây khung nhỏ nhất (minimum spanning tree)

Thuật toán cải tiến lặp (Iterative improvement algorithms)

Trong nhiều bài toán tối ưu hóa, *đường đi* không quan trọng;
bản thân trạng thái đích chính là giải pháp

Khi đó, không gian trạng thái = tập các cấu hình “đầy đủ”;

tìm cấu hình *tối ưu*, ví dụ TSP

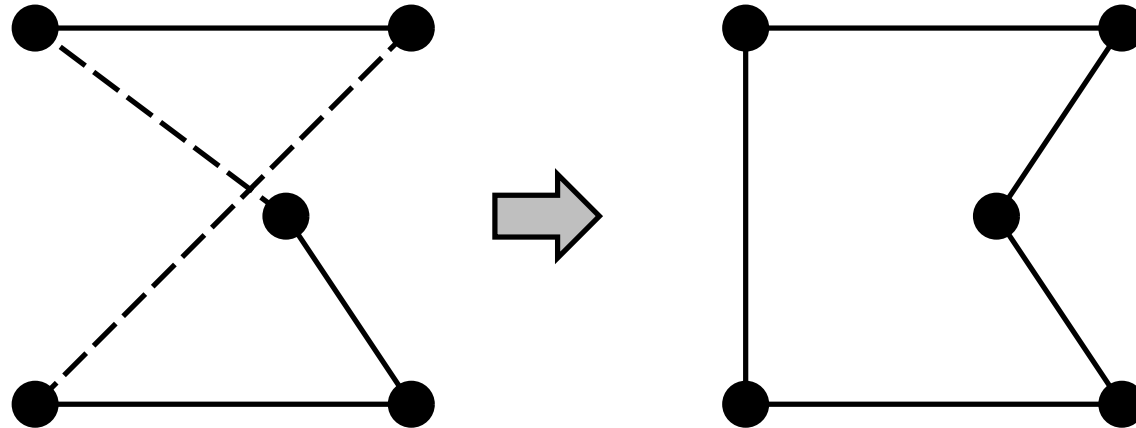
hoặc tìm cấu hình thỏa mãn ràng buộc, ví dụ n-quân hậu

Trong những trường hợp này, có thể sử dụng các thuật toán *cải tiến lặp*;
giữ một trạng thái “hiện tại” duy nhất, và cố gắng cải thiện nó

Không gian bộ nhớ không đổi, thích hợp cho cả tìm kiếm trực tuyến và ngoại tuyến

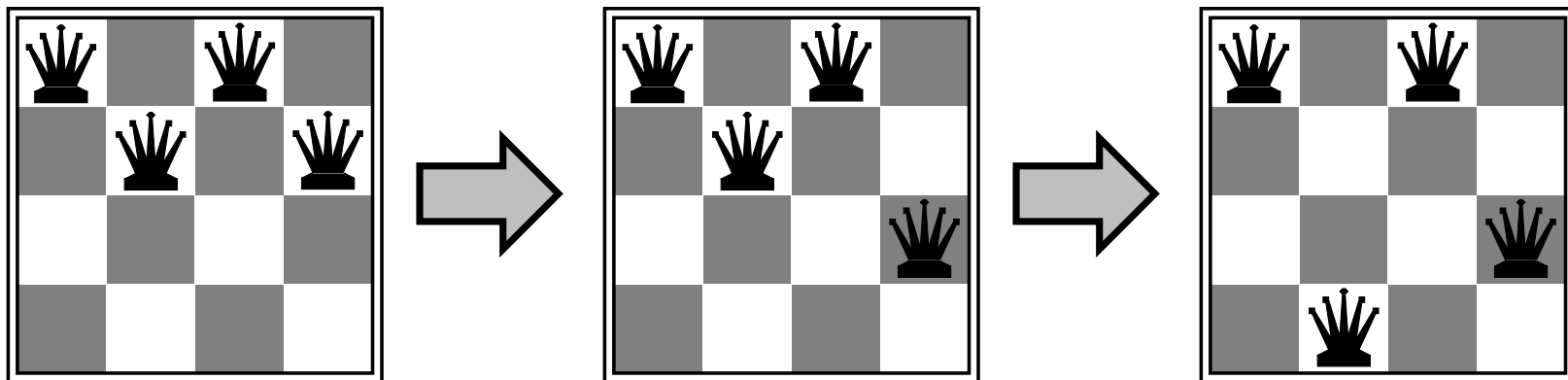
Ví dụ: Bài toán Người giao hàng (TSP)

Tìm hành trình ngắn nhất đi qua mỗi thành phố đúng một lần



Ví dụ: Bài toán n -quân hậu

Đặt n quân hậu trên bàn cờ $n \times n$ sao cho không có hai quân hậu nào nằm trên cùng một hàng, cột, hoặc đường chéo



Leo đồi (Hill-climbing / gradient ascent / descent)

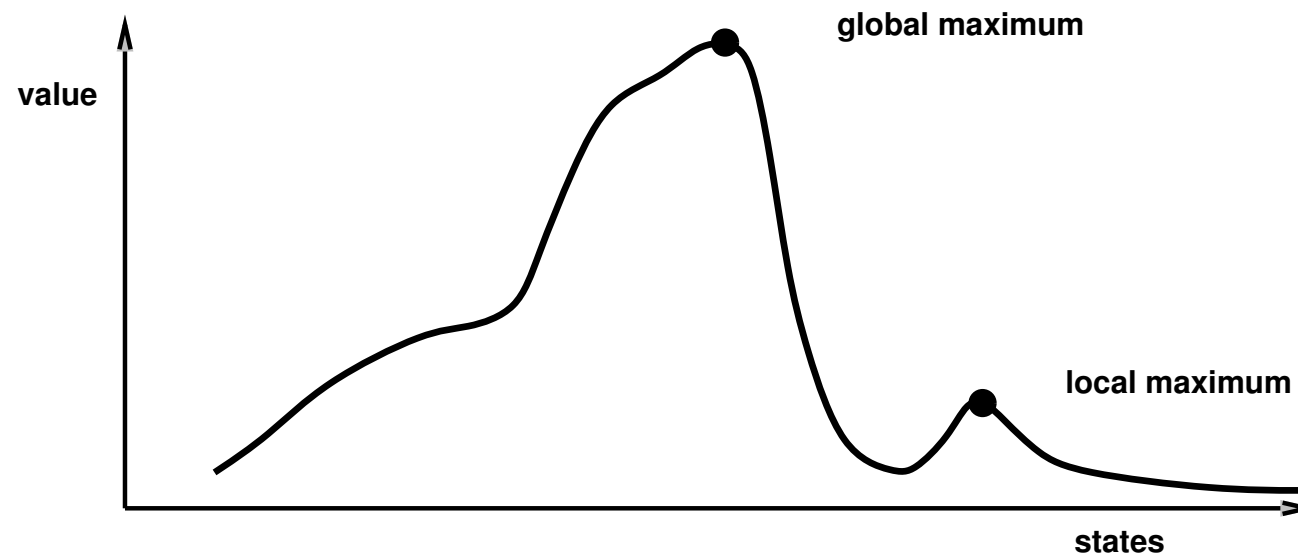
“Giống như leo đỉnh Everest trong màn sương mù dày đặc với chúng mất trí nhớ”

```
function HILL-CLIMBING(problem) returns một trạng thái giải pháp
inputs: problem, một bài toán
local variables: current, một nút
                   next, một nút

current ← MAKE-NODE(INITIAL-STATE[problem])
loop do
  next ← một nút kế tiếp có giá trị cao nhất của current
  if VALUE[next] < VALUE[current] then return current
  current ← next
end
```

Leo đòi (tiếp)

Vấn đề: tùy thuộc vào trạng thái ban đầu, có thể bị mắc kẹt ở cực đại cục bộ (local maxima)



Luyện kim mô phỏng (Simulated annealing)

Ý tưởng: thoát khỏi cực đại cục bộ bằng cách cho phép một số bước đi “tồi”

nhưng giảm dần kích thước và tần suất của chúng

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** một trạng thái giải pháp

inputs: *problem*, một bài toán

schedule, một ánh xạ từ thời gian sang “nhiệt độ”

local variables: *current*, một nút

next, một nút

T, một “nhiệt độ” kiểm soát xác suất của các bước đi xuống

current $\leftarrow$ MAKE-NODE(INITIAL-STATE[*problem*])

for *t* $\leftarrow$ 1 **to** ∞ **do**

T $\leftarrow$ *schedule*[*t*]

if *T*=0 **then return** *current*

next $\leftarrow$ một nút kế tiếp được chọn ngẫu nhiên của *current*

$\Delta E \leftarrow$ VALUE[*next*] – VALUE[*current*]

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

else *current* $\leftarrow$ *next* chỉ với xác suất $e^{\Delta E/T}$

Thuộc tính của luyện kim mô phỏng

Ở một “nhiệt độ” cố định T , xác suất chiếm hữu trạng thái đạt đến phân bố Boltzman

$$p(x) = \alpha e^{\frac{E(x)}{kT}}$$

T giảm đủ chậm $\implies$ luôn đạt được trạng thái tốt nhất

“Đây có nhất thiết là một sự đảm bảo thú vị không???”

Được phát triển bởi Metropolis và cộng sự vào năm 1953, dùng cho mô phỏng quá trình vật lý

Được sử dụng rộng rãi trong thiết kế bố trí VLSI, lập lịch hàng không, v.v.